

# Conceptos básicos

## 1. Naturaleza de GIT

- **¿Qué es GIT?** : Git es un sistema de control de versiones distribuido, es decir, una herramienta que permite registrar, organizar y gestionar los cambios que se realizan en archivos a lo largo del tiempo, especialmente en proyectos de software, funcionando como un historial inteligente que guarda "fotografías" (snapshots) del estado completo del proyecto cada vez que se realiza un cambio importante (commit), incluyendo qué se modificó, quién lo hizo, cuándo y por qué. Esto permite volver a versiones anteriores, comparar cambios y trabajar sin riesgo de perder información, al ser distribuido, cada usuario tiene una copia completa del proyecto y su historial en su propia máquina, lo que facilita el trabajo sin depender de un servidor central y mejora la colaboración entre varias personas.
- **Sistema de control de versiones distribuido** : Git es un tipo de sistema donde cada usuario posee una copia completa del proyecto junto con todo su historial de cambios en su propia máquina, lo que permite trabajar de forma independiente sin conexión constante a un servidor central y facilita la colaboración sin depender de un único punto de fallo.
- **Basado en snapshots (no solo cambios lineales de archivos)** : Git no guarda únicamente las diferencias entre versiones, sino que registra el estado completo del proyecto en cada cambio (como una "fotografía"), lo que hace más eficiente el acceso a versiones pasadas y simplifica la gestión del historial.
- **Historial inmutable** : En Git una vez que un cambio (commit) es registrado, no se modifica directamente, en su lugar, cualquier cambio posterior genera un nuevo registro, lo que garantiza la integridad y trazabilidad del historial del proyecto.
- **Identificación de objetos mediante hashes SHA-1** : Git utiliza funciones hash (SHA-1) para asignar un identificador único a cada elemento almacenado (commits, archivos, etc.), lo que asegura que cualquier modificación, por mínima que sea, genere un identificador distinto y permita verificar la integridad de los datos.

## 2. Componentes del modelo

- **Working Directory** : Estado actual de los archivos en la carpeta de tu computadora, aquí es donde se crea, edita o borra archivos directamente. Se usa durante todo el proceso de desarrollo, ya que representa el "taller" donde estás haciendo los cambios en tiempo real antes de guardarlos formalmente.
- **Staging Area (Index)** : Zona intermedia o "borrador" donde se preparan los cambios que se quieren incluir en el próximo guardado. Se usa cuando ya terminaste una tarea específica en un archivo y quieres marcarlo para que Git lo tome en cuenta, permitiéndote seleccionar exactamente qué archivos subir y cuáles dejar fuera del siguiente registro.

- **Repositorio** : Base de datos de Git donde almacena todo el historial de cambios y las versiones de tu proyecto.  
Se usa cuando necesitas consultar el pasado del código, trabajar en equipo o asegurar que el progreso esté guardado de forma permanente y recuperable en una estructura organizada.
- **HEAD** : Puntero o "indicador" que señala en qué punto exacto del historia del proyecto te encuentras actualmente (normalmente apunta al último cambio de la rama activa).  
Se usa automáticamente cada vez que navegas entre versiones o ramas, sirviendo como la marca de "usted está aquí" dentro de tu línea de tiempo.
- **Tracking Branch (Rama de Seguimiento)** : Rama local que está vinculada directamente a una rama de un servidor remoto (como GitHub).  
Se usa para facilitar la sincronización de archivos, permitiendo que Git sepa exactamente cuántos cambios te faltan por subir o bajar respecto al servidor sin tener que escribir rutas largas.
- **Commit** : Captura o "foto" de los cambios que estaban en el Staging Area, guardándose de forma permanente en el Repositorio con un mensaje descriptivo.  
Se usa cada vez que completas una unidad lógica de trabajo (como arreglar un error o añadir una función) para crear un punto de restauración seguro.

### 3. Tipo de repositorios (especificación)

- **Local** : Base de datos de Git que vive físicamente en el disco duro de la computadora (dentro de una carpeta oculta .git).  
Se usa para trabajar de forma autónoma y privada, permitiendo guardar versiones (commits), crear ramas y revisar el historial de cambios sin necesidad de tener una conexión a internet.
- **Remoto** : Versión de un proyecto alojada en un servidor externo o en la nube (como GitHub, GitLab o Bitbucket).  
Se usa como punto de encuentro para el trabajo en equipo, para respaldar tu código fuera de tu computadora personal y para compartir las versiones finales o avances con otros desarrolladores, algunos ejemplos de servidores externos pueden ser GitHub, GitLab o Bitbucket, útiles porque puedes subir contenido a tu repositorio local, sincronizar cambios y trabajar en equipo de forma centralizada aunque Git sea distribuido.

### 4. Estado y seguimiento de cambios

- **git status** : Muestra el estado actual de tu directorio de trabajo y del área de preparación (staging).  
Se usa cada vez que quieres saber qué archivos has modificado, cuáles están listos para guardarse y qué archivos Git aún no está rastreando, siendo la brújula principal antes de cualquier otra acción.
- **git add** . : Transfiere los cambios de un archivo específico desde tu directorio de trabajo al área de preparación (staging).

Se usa cuando has terminado de editar un archivo y quieres marcarlo como "listo" para ser incluido en la siguiente foto o versión de tu proyecto.

- **git add -p** : Abre un modo interactivo que divide los cambios de tus archivos en pequeños fragmentos (llamados "hunks") y te pregunta, uno por uno, si quieres enviarlos al área de preparación.  
Se usa cuando has trabajado en varias cosas distintas dentro de un mismo archivo y quieres separar esos cambios en diferentes commits para que el historial sea más ordenado y fácil de leer.
- **git commit -m ""** : Crea un guardado permanente en el repositorio local con todos los cambios que estaban en el área de preparación, adjuntando una nota explicativa. Se usa para cerrar una tarea o mejora específica, dejando una marca en la historia del proyecto con una descripción de qué se hizo.
- **git commit -a -m ""** : Realiza un guardado automático de todos los archivos que ya estaban bajo seguimiento de Git y fueron modificados, saltándose el paso manual del "add".  
Se usa para ahorrar tiempo cuando sabes que quieres guardar absolutamente todos tus cambios recientes en un solo paso rápido.
- **git commit --amend** : Reemplaza el último commit realizado por uno nuevo que incluya los cambios actuales del área de preparación.  
Se usa cuando olvidas añadir un archivo al último guardado o cuando quieres corregir un error en el mensaje del commit anterior sin generar uno nuevo.
- **git commit -p** : Combina los pasos de seleccionar fragmentos y crear un guardado en un solo comando, permitiendo elegir qué partes del código modificado quieres registrar directamente en el repositorio.  
Se usa para ahorrar tiempo cuando quieres hacer un "commit" quirúrgico de solo ciertas líneas de código, saltándote el paso manual de preparar los archivos primero.
- **git restore** : Revierte los cambios realizados en un archivo del directorio de trabajo, devolviéndolo al estado de su último guardado.  
Se usa cuando cometes un error editando un archivo y quieres descartar todo lo que hiciste para volver a la versión segura que ya tenías.
- **git restore --staged** : Saca un archivo del área de preparación (staging) pero mantiene los cambios intactos en tu carpeta de trabajo.  
Se usa cuando agregas un archivo por error con "git add" y decides que aún no estás listo para guardarlo en el próximo commit.

## 5. Archivos ignorados

- **.gitignore** : Archivo de configuración que indica a Git qué archivos o directorios debe ignorar, evitando que sean rastreados o incluidos en commits.  
Se utiliza comúnmente para excluir archivos temporales, configuraciones locales, dependencias o datos generados automáticamente.

- **git status --ignored** : Muestra el estado del repositorio incluyendo también los archivos que están siendo ignorados por el “.gitignore”, permitiendo ver explícitamente qué elementos no están siendo rastreados por Git.

## 6. Inicialización y clonación

- **git init** : Crea un nuevo repositorio de Git vacío en la carpeta actual, generando el directorio oculto “.git” donde se guardará todo el historial futuro.  
Se usa al comenzar un proyecto desde cero en tu computadora para que Git empiece a rastrear cada cambio que hagas en tus archivos desde ese momento.
- **git clone <URL>** : Descarga una copia completa de un repositorio ya existente en un servidor remoto (como GitHub) hacia una carpeta nueva en tu computadora.  
Se usa cuando quieres empezar a trabajar en un proyecto que ya tiene historia o cuando te unes a un equipo de desarrollo y necesitas tener exactamente los mismos archivos que ellos.

## 7. Exploración del historial

- **git log** : Muestra el historial completo de commits realizados en la rama actual, incluyendo autor, fecha y descripción detallada.  
Se usa cuando se necesita revisar la cronología completa del proyecto y entender quién hizo qué cambios a lo largo del tiempo.
- **git log --oneline** : Resume cada commit en una sola línea, mostrando solo el código identificador corto y el mensaje descriptivo.  
Se usa para obtener una vista rápida y compacta de muchos commits a la vez, ideal para buscar un cambio específico sin saturar la pantalla.
- **git log --graph** : Presenta el historial de forma resumida, mostrando cada commit en una sola línea con su hash abreviado y mensaje, lo que hace es dibujar una representación visual con líneas y nodos que muestra cómo se bifurcan y se unen las distintas ramas del proyecto.  
Se usa para entender la estructura compleja del repositorio, especialmente cuando varias personas trabajan en ramas diferentes simultáneamente.
- **git log --decorate** : Añade etiquetas visuales a los commits para mostrar a qué rama pertenecen o si tienen etiquetas (tags) especiales.  
Se usa cuando quieres saber exactamente dónde están situadas las referencias de las ramas y el puntero HEAD dentro de tu historial.
- **git log --follow archivo.txt** : Rastrea y muestra el historial de un archivo específico, incluso si este ha sido renombrado en el pasado. Se usa cuando necesitas conocer la evolución completa de un archivo sin perder el rastro de sus cambios antiguos debido a un cambio de nombre.
- **git log archivo.txt** : Filtra el historial para mostrar únicamente los commits que afectaron a un archivo en particular.  
Se usa cuando se quiere investigar por qué cambió un archivo específico o quién fue el responsable de las modificaciones en esa pieza de código.

- **git log -S " ":** Busca dentro del contenido de los commits para encontrar cuándo se añadió o eliminó una cadena de texto específica.  
Se usa como una herramienta de "detective" para localizar el momento exacto en que una función, variable o error fue introducido en el código.
- **git shortlog :** Agrupa los commits por autor y muestra un conteo de cuántos ha realizado cada uno. Se usa para generar resúmenes rápidos de contribuciones, ideal para reportes de avance o para ver quién ha estado más activo en el desarrollo.
- **git blame :** Muestra el contenido de un archivo línea por línea, indicando quién fue la última persona que modificó cada una y en qué commit lo hizo.  
Se usa para identificar al responsable de una línea de código específica, ya sea para pedirle contexto o para entender cuándo se rompió algo.
- **git blame -L 10,20 :** Aplica el último comando anteriormente usado pero limitado a un rango específico de líneas dentro de un archivo.  
Se usa para analizar rápidamente un bloque de código concreto sin tener que leer el historial de todo el documento, ahorrando tiempo en archivos muy extensos.
- **git show <HASH> :** Muestra el contenido detallado de un commit específico, incluyendo el mensaje, el autor y exactamente qué líneas de código se añadieron o borraron (el diff).  
Se usa cuando tienes el identificador de un cambio y necesitas ver con precisión quirúrgica qué se hizo en ese momento.

## 8. Gestión de ramas

- **git branch :** Crea una nueva rama a partir de un punto específico en el pasado, usando el código identificador de un commit antiguo.  
Se usa cuando necesitas "viajar en el tiempo" para retomar el desarrollo desde un estado anterior del código sin afectar lo que se ha hecho después.
- **git branch -r :** Muestra una lista de todas las ramas que existen en el servidor remoto (como GitHub) pero que no necesariamente has descargado a tu computadora.  
Se usa cuando necesitas saber qué están haciendo tus compañeros en otras ramas antes de decidir si quieres traer esos cambios a tu entorno local.
- **git branch -a :** Lista absolutamente todas las ramas que conoce Git, combinando tanto las locales como las remotas en una sola vista.  
Se usa para obtener un panorama completo de la estructura del proyecto y verificar si una rama existe en algún lugar del ecosistema de trabajo.
- **git branch -m :** Renombra la rama en la que te encuentras actualmente por un nombre nuevo que tú elijas.  
Se usa cuando el nombre original de la rama ya no describe bien la tarea que estás realizando o para corregir errores de escritura en los nombres de las ramas.

- **git switch** : Cambia de una rama a otra dentro de tu repositorio, actualizando los archivos de tu carpeta de trabajo para que coincidan con la rama seleccionada. Se usa constantemente para saltar entre diferentes tareas o funciones del proyecto sin mezclar el código de una con la otra.
- **git switch -c** : Crea una rama nueva y te mueve a ella inmediatamente en un solo paso. Se usa en el momento exacto en que decides empezar una nueva mejora o corrección, ahorrándote el paso de crear la rama y luego cambiarte manualmente.
- **git switch --track origin/<rama>** : Crea una copia local de una rama que solo existe en el servidor remoto y establece una conexión directa entre ambas. Se usa cuando quieres empezar a trabajar en una rama que un compañero ya subió al servidor para poder sincronizar tus avances con los de él fácilmente.

## 9. Fusión y rebase

- **git merge** : Une la historia de una rama secundaria dentro de tu rama actual, creando un nuevo "commit de fusión" que conecta ambos caminos. Se usa cuando has terminado una funcionalidad en una rama aparte y quieres integrar todo ese trabajo de manera definitiva en la rama principal del proyecto.
- **git merge --abort** : Detiene el proceso de fusión actual y devuelve tus archivos exactamente al estado en que estaban antes de intentar el merge. Se usa únicamente cuando surgen conflictos de código que no sabes cómo resolver en ese momento o si te das cuenta de que estás fusionando las ramas equivocadas.
- **git rebase** : Reaplica tus commits actuales uno por uno sobre la punta de otra rama, "reescribiendo" la historia para que parezca una línea recta sin curvas de fusión. Se usa para mantener un historial limpio y lineal, integrando los cambios más recientes de otros compañeros antes de enviar tu propio trabajo al servidor.
- **git rebase --abort** : Cancela por completo la operación de rebase que estaba en curso y restaura tu rama a su estado original. Se usa cuando el proceso de rebase se vuelve demasiado complejo debido a múltiples conflictos y prefieres empezar de cero o intentar otra estrategia de integración.
- **git rebase --continue** : Reanuda el proceso de rebase después de que hayas resuelto manualmente un conflicto en un archivo y lo hayas marcado con "git add". Se usa paso a paso durante un rebase para avanzar al siguiente commit de la lista hasta finalizar toda la integración.
- **git pull --rebase** : Descarga los cambios nuevos del servidor y, en lugar de fusionarlos con un merge, mueve tus cambios locales para que queden por encima de lo que acabas de descargar. Se usa para evitar que se creen "commits de fusión" innecesarios cada vez que actualizas tu código con el trabajo de los demás, manteniendo la historia del proyecto mucho más clara.

## 10. Operaciones remotas

- **git push origin** : Envía los commits locales desde la rama actual hacia el servidor remoto (origin).  
Se usa cuando has terminado una tarea y quieres que tus cambios estén disponibles para el resto del equipo o respaldados en la nube.
- **git push -u origin** : Sube tus cambios al servidor y establece una conexión permanente entre tu rama local y la remota.  
Se usa la primera vez que envías una rama nueva al servidor, permitiendo que en el futuro solo tengas que escribir "git push" o "git pull" sin especificar nombres.
- **git pull origin** : Descarga los cambios más recientes del servidor remoto y los fusiona automáticamente con tu código actual.  
Se usa al empezar tu jornada de trabajo para asegurarte de tener la versión más actualizada de lo que han hecho tus compañeros.
- **git push --force-with-lease** : Sobreescribe el historial del servidor remoto con tus cambios locales, pero solo si nadie más ha subido cambios nuevos mientras tanto.  
Se usa después de hacer un "rebase" para limpiar el historial, siendo una forma segura de forzar una actualización sin borrar accidentalmente el trabajo de otros.
- **git fetch** : Consulta el servidor remoto para ver si hay cambios nuevos, pero no los aplica a tus archivos de trabajo.  
Se usa cuando quieres revisar lo que han hecho los demás sin arriesgarte a que se desordene o se rompa tu código actual.
- **git fetch --prune** : Actualiza tu lista de ramas remotas y borra de tu computadora las referencias a ramas que ya fueron eliminadas en el servidor. Se usa para mantener limpio tu listado de ramas y no ver nombres de tareas que ya terminaron y se borraron de GitHub.
- **git remote** : Muestra la lista de repositorios remotos configurados en el repositorio local.
- **git remote remove** : Elimina el vínculo o conexión entre tu carpeta local y un servidor remoto específico. Se usa cuando decides dejar de enviar código a un servidor antiguo o cuando quieres reconfigurar desde dónde descargas el proyecto.
- **git remote -v** : Muestra las direcciones URL exactas de los servidores remotos con los que tu proyecto está conectado.  
Se usa para verificar que estás enviando y recibiendo el código desde el lugar correcto (como por ejemplo el repositorio oficial de una empresa).
- **git remote add** : Establece una conexión nueva entre el repositorio local y un servidor en internet dándole un nombre (usualmente "origin").  
Se usa al inicio de un proyecto para decirle a tu computadora a qué dirección de GitHub o GitLab debe enviar el código.

- **git remote prune origin** : Borra de tu rastreo local todas las ramas que ya no existen en el servidor "origin". Se usa específicamente para sincronizar tu vista de ramas con la realidad del servidor, eliminando "ramas fantasma" que ya fueron borradas por otros.

## 11. Eliminación y renombrado de archivos

- **git rm** : Elimina un archivo tanto de tu carpeta de trabajo (disco duro) como del área de preparación de Git.  
Se usa cuando decides que un archivo ya no es necesario en el proyecto y quieres que desaparezca físicamente y deje de ser rastreado en el próximo commit.
- **git rm --cached** : Elimina un archivo del área de preparación y del control de versiones de Git, pero lo mantiene físicamente intacto en tu computadora.  
Se usa cuando subiste por error un archivo que no debería estar en el repositorio (como una contraseña o una carpeta de configuración personal) y quieres que Git lo ignore sin borrarlo de tu equipo.
- **git mv** : Cambia el nombre de un archivo o lo mueve a una carpeta distinta, informando a Git del cambio inmediatamente.  
Se usa para reorganizar la estructura de tu proyecto sin perder el historial de cambios del archivo, evitando que Git lo interprete como si hubieras borrado un archivo y creado uno nuevo desde cero.

## 12. Configuración inicial

- **git config --global user.name** : Establece el nombre de usuario que Git asociará a los commits realizados desde el equipo actual.
- **git config --global user.email** : Establece la dirección de correo electrónico que Git asociará a los commits realizados desde el equipo actual.
- **git config --list** : Muestra todas las configuraciones activas de Git, incluyendo las definidas a nivel del sistema, usuario y repositorio.
- **git config --global init.defaultBranch** : Define el nombre de la rama principal que Git utilizará por defecto al crear nuevos repositorios mediante `git init`.